Phishing URL Detector

Complete Source Code Documentation

Dual Machine-Learning Engine · Flask Web Application

Logistic Regression + Random Forest · Real-time Risk Analysis

Batch URL analysis · LR / RF metrics · Per-model detection reasons

Python 3 · HTML5 · CSS3 · JavaScript · scikit-learn · Flask

For educational & research use only

1. Project Overview

About This Project

A full-stack phishing URL detection system built with Python and Flask. Two independent ML models (Logistic Regression and Random Forest) analyse each URL separately — producing their own risk score, verdict, and explanation. Users can submit one URL or a batch of up to 100 URLs. Results are combined into overall verdicts in the responsive web UI. The system also compares both models using 5-fold stratified cross-validation on its 20 synthetic sample URLs.

Tech Stack

Language	Python 3.13
Web Framework	Flask 3.x
ML Models	Logistic Regression + Random Forest (scikit-learn)
Data	pandas, numpy — 20 training samples, 6 features
Frontend	HTML5, CSS3 (dark theme), Vanilla JavaScript (ES2020)
Blacklist	blacklist.json — auto-generated on startup

File Structure

phishing_detector.py	Flask server, ML training, full prediction pipeline
templates/index.html	Full page UI: hero, awareness tips, workflow, analyzer
static/style.css	Dark-theme stylesheet for all UI components
static/app.js	Frontend: fetch calls, per-model result rendering
blacklist.json	Known-bad domains (auto-created at startup)

Detection Workflow

Step 1 — URL Input	User enters any URL, domain, or IP link
Step 2 — Normalize	Strip whitespace, prepend https:// if missing
Step 3 — Blacklist Check	Domain checked against known-bad list → instant PHISHING if matched
Step 4 — Feature Extraction	url_length, domain_length, qty_dot, qty_hyphen, is_ip, has_at
Step 5a — Logistic Regression	LR probability + LR verdict + LR-specific reasons
Step 5b — Random Forest	RF probability + RF verdict + RF-specific reasons
Step 6 — Aggregate	Combined score = average of LR and RF probabilities
Step 7 — Verdict	< 35% = SAFE | 35–70% = SUSPICIOUS | > 70% = PHISHING

API Response Fields

status	Overall verdict: SAFE / SUSPICIOUS / PHISHING
risk_score	Combined averaged risk percentage
lr_prob	Logistic Regression risk %
lr_status	LR individual verdict
lr_reasons	List of LR-specific reasons (feature weight explanations)
rf_prob	Random Forest risk %
rf_status	RF individual verdict
rf_reasons	List of RF-specific reasons (ensemble voting explanations)
reasons	Shared structural detection insights
count / results	Batch response fields: the number of URLs and one prediction per URL
/metrics	5-fold cross-validation Accuracy, Precision, Recall, and F1-Score for both models

2. phishing_detector.py — Python Backend

■ phishing_detector.py

Python 3 · Flask · scikit-learn

■ Imports and project setup

```
"""
Phishing URL Detector — Flask Application
Combines model training, prediction pipeline, and web serving.
Returns separate risk scores for Logistic Regression and Random Forest.
"""

import os
import re
import json
import warnings
import numpy as np
import pandas as pd
from urllib.parse import urlparse
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_predict
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from flask import Flask, request, jsonify, render_template, send_file

warnings.filterwarnings("ignore")

# ------------------------------------------------------------------------
```

■ 1 — Blacklist configuration

```
# 1. Blacklist setup
# ------------------------------------------------------------------------
BLACKLIST_DOMAINS = [
    "paypal-security-update.com",
    "login-secure-bank.net",
    "verify-amazon-account.info",
]
with open("blacklist.json", "w") as f:
    json.dump(BLACKLIST_DOMAINS, f, indent=4)


# ------------------------------------------------------------------------
```

■ 2 — Feature extraction

```
# 2. Feature extraction
# ---------------------------------------------------------------------------
FEATURE_NAMES = ["url_length", "domain_length", "qty_dot", "qty_hyphen", "is_ip", "has_at"]

def extract_features(url: str) -> dict:
    """Extract numeric features from a URL for ML classification."""
    parsed = urlparse(url)
    domain = parsed.netloc if parsed.netloc else parsed.path
    return {
        "url_length":    len(url),
        "domain_length": len(domain),
        "qty_dot":       url.count("."),
        "qty_hyphen":    url.count("-"),
        "is_ip":         1 if re.match(r"^\d{1,3}(\.\d{1,3}){3}$", domain) else 0,
        "has_at":        1 if "@" in url else 0,
    }


# ---------------------------------------------------------------------------
```

■ 3 — Training data, models, and cross-validation

```
# 3. Training data & model fitting
# ------------------------------------------------------------------------
TRAINING_DATA = [
    # --- Safe URLs (label 0) ---
    ("https://www.google.com",        0),
    ("https://www.github.com",        0),
    ("https://www.wikipedia.org",     0),
    ("https://www.nytimes.com",       0),
    ("https://www.amazon.com",        0),
    ("https://www.microsoft.com",     0),
    ("https://www.apple.com",         0),
    ("https://www.reddit.com",        0),
    ("https://www.linkedin.com",      0),
    ("https://www.stackoverflow.com", 0),
    # --- Phishing / suspicious URLs (label 1) ---
    ("https://login-secure-paypal-verification.com/webscr",  1),
    ("http://verify-your-bank-account-now.info/update",      1),
    ("http://192.168.1.105/index.php?login=true",            1),
    ("https://amazon-security-alert-center.com/login",       1),
    ("http://secure-update-signin@hotmail-verify.com",       1),
    ("http://get-free-coins-fast.xyz/claim",                 1),
    ("https://netflix-account-onhold.net/renew",             1),
    ("http://10.0.0.1/admin-login",                          1),
    ("https://apple-id-locked-verify.com/account",           1),
    ("http://free-gift-card-generator.online/redeem",        1),
]

X = pd.DataFrame([extract_features(url) for url, _ in TRAINING_DATA])
y = np.array([label for _, label in TRAINING_DATA])

lr_model = LogisticRegression(max_iter=1000)
lr_model.fit(X, y)

rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X, y)

_cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
MODEL_METRICS = {}
for _name, _model in (("Logistic Regression", LogisticRegression(max_iter=1000)),
                      ("Random Forest", RandomForestClassifier(n_estimators=100, random_state=42))):
    _predictions = cross_val_predict(_model, X, y, cv=_cv)
    MODEL_METRICS[_name] = {
        "accuracy": round(accuracy_score(y, _predictions) * 100, 1),
        "precision": round(precision_score(y, _predictions, zero_division=0) * 100, 1),
        "recall": round(recall_score(y, _predictions, zero_division=0) * 100, 1),
        "f1_score": round(f1_score(y, _predictions, zero_division=0) * 100, 1),
    }

print("Models trained -- LR + RandomForest ready.")
print(f"   Training samples: {len(TRAINING_DATA)}  |  Features: {FEATURE_NAMES}")


# ------------------------------------------------------------------------
```

■ 4 — Prediction and explanation helpers

```
# 4. Prediction helpers
# ------------------------------------------------------------------------
def normalize_url(url: str) -> str:
    """Ensure URL has a scheme so urlparse works correctly."""
    url = url.strip()
    if not url.lower().startswith(("http://", "https://")):
        url = "https://" + url
    return url


def check_blacklist(url: str) -> bool:
    """Return True if the URL's domain matches a known-bad entry."""
    domain = urlparse(url).netloc.lower()
    try:
        with open("blacklist.json") as f:
            blacklist = json.load(f)
    except FileNotFoundError:
        blacklist = []
    return any(bad in domain for bad in blacklist)


def classify(pct: float) -> str:
    """Map a risk percentage to a verdict label."""
    if pct < 35:
        return "SAFE"
    if pct < 70:
        return "SUSPICIOUS"
    return "PHISHING"


def build_reasons(feats: dict) -> list[str]:
    """Generate shared human-readable detection insights from extracted features."""
    reasons = []
    if feats["url_length"] > 50:
        reasons.append(f"Abnormally long URL ({feats['url_length']} characters) — often used to hide the
    real destination.")
    if feats["qty_hyphen"] >= 2:
        reasons.append("Multiple hyphens in the domain — a common trick to mimic legitimate brand names
    (e.g. paypal-secure-login.com).")
    if feats["is_ip"]:
        reasons.append("An IP address is used instead of a domain name — legitimate sites almost never do
    this.")
    if feats["has_at"]:
        reasons.append("The '@' symbol was detected — browsers ignore everything before '@', redirecting you
    to a different site.")
    if feats["qty_dot"] > 4:
        reasons.append(f"Excessive subdomains ({feats['qty_dot']} dots) — attackers add layers to make a URL
    look official.")
    if not reasons:
        reasons.append("No structural anomalies detected. URL structure appears normal.")
    return reasons


def build_lr_reasons(feats: dict, prob: float) -> list[str]:
    """
    Logistic Regression-specific reasons.
    LR is a linear model — it weights each feature independently.
    Explain which features pushed the linear score high or low.
    """
    reasons = []

    # High risk contributors
    if prob >= 70:
        if feats["qty_hyphen"] >= 2:
```

```
            reasons.append(
                f"LR heavily weighted the {feats['qty_hyphen']} hyphens — "
                "this single feature has a strong positive coefficient for phishing in a linear model."
            )
        if feats["url_length"] > 50:
            reasons.append(
                f"URL length ({feats['url_length']} chars) pushed the LR score up — "
                "the model learned that longer URLs correlate with obfuscation attempts."
            )
        if feats["has_at"]:
            reasons.append(
                "The '@' symbol carries a very high LR coefficient — "
                "it is almost exclusively seen in malicious URLs."
            )
        if feats["is_ip"]:
            reasons.append(
                "Raw IP address in the URL has a strong positive weight in the LR model — "
                "no legitimate brand uses a bare IP as its address."
            )
        if feats["qty_dot"] > 4:
            reasons.append(
                f"High dot count ({feats['qty_dot']}) raised the LR probability — "
                "deep subdomain nesting is a known phishing pattern."
            )
        if not reasons:
            reasons.append(
                f"The combined linear feature weights produced a high phishing probability ({prob}%). "
                "No single dominant feature, but the overall URL profile matches phishing patterns."
            )

    # Medium risk
    elif prob >= 35:
        if feats["qty_hyphen"] == 1:
            reasons.append(
                "One hyphen in the domain slightly raised the LR score — "
                "not conclusive alone, but combined with other features it adds suspicion."
            )
        if feats["url_length"] > 30:
            reasons.append(
                f"Moderately long URL ({feats['url_length']} chars) contributed a partial positive weight in
LR."
            )
        if feats["qty_dot"] >= 3:
            reasons.append(
                f"Three or more dots ({feats['qty_dot']}) added a mild positive weight — "
                "LR sees this as a weak phishing signal."
            )
        if not reasons:
            reasons.append(
                f"LR produced a borderline score ({prob}%) — "
                "no strongly weighted features, but the URL profile is not entirely clean."
            )

    # Low risk
    else:
        reasons.append(
            f"LR score is low ({prob}%) — the URL's features (short length, no hyphens, "
            "no IP, no '@') align closely with the safe URLs in training data."
        )
        if feats["domain_length"] <= 15:
            reasons.append(
                f"Short domain length ({feats['domain_length']} chars) is a negative indicator for phishing
"
                "in the LR model — legitimate brands tend to have concise domains."
            )

    return reasons
```

```
def build_rf_reasons(feats: dict, prob: float) -> list[str]:
    """
    Random Forest-specific reasons.
    RF uses an ensemble of decision trees — it captures non-linear interactions.
    Explain the tree-based decision logic.
    """
    reasons = []

    # High risk
    if prob >= 70:
        if feats["qty_hyphen"] >= 2 and feats["url_length"] > 40:
            reasons.append(
                f"The RF trees found the combination of {feats['qty_hyphen']} hyphens AND "
                f"long URL ({feats['url_length']} chars) together — this interaction is a strong "
                "phishing pattern the ensemble consistently splits on."
            )
        elif feats["qty_hyphen"] >= 2:
            reasons.append(
                f"Multiple hyphens ({feats['qty_hyphen']}) triggered a majority of RF decision trees "
                "to classify this as phishing — the ensemble reached high agreement."
            )
        if feats["is_ip"]:
            reasons.append(
                "Every RF tree that encountered an IP-address flag voted PHISHING — "
                "the ensemble is unanimous on this feature."
            )
        if feats["has_at"]:
            reasons.append(
                "The '@' flag caused near-unanimous agreement across RF trees — "
                "this pattern does not appear in any safe URL in the training set."
            )
        if feats["qty_dot"] > 4:
            reasons.append(
                f"Deep subdomain nesting ({feats['qty_dot']} dots) caused most RF trees to "
                "branch toward the phishing leaf node."
            )
        if not reasons:
            reasons.append(
                f"The RF ensemble voted {prob}% confidence for PHISHING — "
                "multiple trees independently identified this URL's feature combination as suspicious."
            )

    # Medium risk
    elif prob >= 35:
        reasons.append(
            f"The RF ensemble was split ({prob}% phishing confidence) — "
            "some trees voted safe, others voted phishing, indicating ambiguous features."
        )
        if feats["qty_hyphen"] == 1:
            reasons.append(
                "One hyphen caused a subset of RF trees to branch toward phishing — "
                "not enough for consensus but enough to raise the aggregate score."
            )
        if feats["url_length"] > 30:
            reasons.append(
                f"URL length of {feats['url_length']} chars pushed some trees toward the phishing branch, "
                "but others considered it within normal range."
            )
        if not reasons:
            reasons.append(
                "The RF trees found mixed signals — the URL has some characteristics of both "
                "safe and phishing URLs in the training data."
            )

    # Low risk
    else:
        reasons.append(
```

```
            f"The RF ensemble voted {100 - prob}% confidence for SAFE — "
            "the majority of decision trees reached the safe leaf node."
        )
        if feats["domain_length"] <= 15 and feats["qty_hyphen"] == 0:
            reasons.append(
                "Short domain with no hyphens — this combination consistently routes "
                "to safe leaf nodes across the RF ensemble."
            )
        if feats["qty_dot"] <= 2:
            reasons.append(
                f"Low dot count ({feats['qty_dot']}) kept RF trees on the safe branch — "
                "minimal subdomain nesting is a hallmark of legitimate URLs."
            )

    return reasons


# ---------------------------------------------------------------------------
```

■ 5 — Prediction pipeline and startup checks

```
# 5. Main prediction function
# ---------------------------------------------------------------------------
def predict_phishing(url: str) -> dict:
    """
    Full pipeline. Returns:
      status        — overall verdict (SAFE / SUSPICIOUS / PHISHING)
      risk_score    — combined (averaged) risk %
      lr_prob       — Logistic Regression risk %
      lr_status     — LR individual verdict
      rf_prob       — Random Forest risk %
      rf_status     — RF individual verdict
      reasons       — list of human-readable insights
    """
    normalized = normalize_url(url)

    # --- Blacklist fast-path ---
    if check_blacklist(normalized):
        return {
            "status":     "PHISHING",
            "risk_score": 100.0,
            "lr_prob":    None,
            "lr_status":  None,
            "rf_prob":    None,
            "rf_status":  None,
            "reasons": [
                "Domain matched a known phishing entry in the system blacklist.",
                "This URL has been reported as malicious. Do not visit it.",
            ],
        }

    # --- Feature extraction ---
    feats   = extract_features(normalized)
    feat_df = pd.DataFrame([feats])

    # --- Individual model probabilities (phishing class = 1) ---
    lr_prob = round(float(lr_model.predict_proba(feat_df)[0][1]) * 100, 1)
    rf_prob = round(float(rf_model.predict_proba(feat_df)[0][1]) * 100, 1)

    # --- Individual verdicts ---
    lr_status = classify(lr_prob)
    rf_status = classify(rf_prob)

    # --- Combined (averaged) score & overall verdict ---
    combined  = round((lr_prob + rf_prob) / 2, 1)
    status    = classify(combined)

    return {
        "status":      status,
        "risk_score":  combined,
        "lr_prob":     lr_prob,
        "lr_status":   lr_status,
        "lr_reasons":  build_lr_reasons(feats, lr_prob),
        "rf_prob":     rf_prob,
        "rf_status":   rf_status,
        "rf_reasons":  build_rf_reasons(feats, rf_prob),
        "reasons":     build_reasons(feats),
    }


# Quick self-test on startup
_tests = [
    ("https://www.google.com",              "SAFE"),
    ("http://paypal-security-update.com",   "PHISHING"),   # blacklist hit
    ("http://login-secure-paypal-verification.com/webscr", "PHISHING"),
```

```
]
print("\n  Self-test results:")
for _url, _expected in _tests:
    _r = predict_phishing(_url)
    _ok = "OK" if _r["status"] == _expected else "FAIL"
    print(f"  {_ok} {_url[:55]:<55}  ->  {_r['status']}  (combined {_r['risk_score']}%)")
# ------------------------------------------------------------------------------
```

■ 6 — Flask routes, batch prediction, and metrics

```
# 6. Flask app
# ---------------------------------------------------------------------------
app = Flask(__name__, template_folder="templates", static_folder="static")


@app.route("/")
def index():
    return render_template("index.html")


@app.route("/pdf")
def serve_pdf():
    pdf_path = os.path.join(os.path.dirname(__file__), "Phishing_URL_Detector_Code.pdf")
    return send_file(pdf_path, mimetype="application/pdf",
                     download_name="Phishing_URL_Detector_Code.pdf")


@app.route("/predict", methods=["POST"])
def predict():
    body = request.get_json(silent=True)
    if not isinstance(body, dict):
        return jsonify({"error": "Send a JSON object containing 'url' or 'urls'."}), 400

    if "urls" in body:
        submitted_urls = body["urls"]
        if isinstance(submitted_urls, str):
            submitted_urls = submitted_urls.splitlines()
        if not isinstance(submitted_urls, list) or any(not isinstance(url, str) for url in submitted_urls):
            return jsonify({"error": "'urls' must be a list of URL strings or newline-separated text."}),
    400
        urls = [url.strip() for url in submitted_urls if url.strip()]
        if not urls:
            return jsonify({"error": "Enter at least one URL."}), 400
        if len(urls) > 100:
            return jsonify({"error": "Analyze up to 100 URLs per request."}), 400
        return jsonify({
            "count": len(urls),
            "results": [{"url": url, **predict_phishing(url)} for url in urls],
        })

    url = body.get("url")
    if not isinstance(url, str) or not url.strip():
        return jsonify({"error": "Missing or empty 'url' field."}), 400
    return jsonify(predict_phishing(url))


@app.route("/metrics")
def metrics():
    return jsonify({
        "models": MODEL_METRICS,
        "sample_count": len(TRAINING_DATA),
        "evaluation": "5-fold stratified cross-validation",
    })


if __name__ == "__main__":
    port = int(os.environ.get("PORT", 5000))
    print(f"Starting server at http://127.0.0.1:{port}")
    app.run(debug=False, host="0.0.0.0", port=port)
```

3. templates/index.html — HTML Frontend

templates/index.html HTML5

Document head and navigation

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <meta name="description" content="WolfSecurity detects phishing links with machine learning and blacklist
    checks." />
  <title>WolfSecurity | Phishing URL Detector</title>
  <link rel="stylesheet" href="/static/style.css" />
</head>
<body>
  <header class="topbar">
    <div class="brand-wrap">
      <div class="brand-mark" aria-label="WolfSecurity wolf logo">
        <svg viewBox="0 0 64 64" aria-hidden="true" focusable="false">
          <path class="wolf-head" d="M11 8 25 15 32 11 39 15 53 8 51 25 59 32 49 47 43 42 39 55 32 60 25 55
    21 42 15 47 5 32 13 25z" />
          <path class="wolf-detail" d="m18 19 10 5-4 5-7-2zm28 0-10 5 4 5 7-2zM23 35l9 5 9-5-4 11-5 4-5-4z"
    />
          <path class="wolf-eye" d="m22 30 7 2-6 3zm20 0-7 2 6 3z" />
        </svg>
      </div>
      <div class="brand-text">
        <span class="brand-name">WolfSecurity</span>
        <span class="brand-tag">Cyber Defense</span>
      </div>
    </div>

    <nav class="main-nav" aria-label="Main navigation">
      <a href="#features">Features</a>
      <a href="#workflow">Workflow</a>
      <a href="#analyzer">Analyzer</a>
      <a href="#about">About</a>
    </nav>

    <a class="nav-cta" href="#analyzer">Analyze URL</a>
  </header>

  <main class="page-wrap">
```

■ Hero

```
<section class="hero">
  <div class="hero-grid">
    <div class="hero-copy">
      <p class="eyebrow">CYBER DEFENSE PLATFORM</p>
      <h1>Detect phishing links before they reach your users.</h1>
      <p class="hero-sub">
        Real-time threat detection powered by a dual machine-learning engine that flags suspicious URLs,
        checks known malicious domains, and explains the risk in plain language.
      </p>

      <div class="hero-actions">
        <a class="primary-btn" href="#analyzer">Try the analyzer</a>
        <a class="secondary-btn" href="#workflow">See how it works</a>
      </div>

      <div class="hero-stats">
        <div class="stat-box">
          <strong>2</strong>
          <span>ML models</span>
        </div>
        <div class="stat-box">
          <strong>100%</strong>
          <span>Blacklist flagging</span>
        </div>
        <div class="stat-box">
          <strong>Live</strong>
          <span>Risk scoring</span>
        </div>
      </div>
    </div>

    <div class="hero-panel">
      <div class="preview-card">
        <div class="preview-header">
          <span class="dot green"></span>
          <span class="dot yellow"></span>
          <span class="dot red"></span>
        </div>

        <div class="scan-box">
          <span class="scan-label">Detected URL</span>
          <strong>paypal-security-update.com</strong>
        </div>

        <div class="scan-score-row">
          <span>Risk score</span>
          <strong>100%</strong>
        </div>

        <div class="mini-bar">
          <span></span>
        </div>

        <ul class="mini-list">
          <li>Known malicious domain</li>
          <li>Suspicious spelling pattern</li>
          <li>Brand impersonation</li>
        </ul>
      </div>
    </div>
  </div>
</section>
```

■ Awareness section

```
<section id="features" class="awareness-banner">
  <div class="section-header">
    <p class="section-kicker">WHY IT MATTERS</p>
    <h2 class="section-title">Cyber awareness starts with smarter detection.</h2>
  </div>

  <div class="awareness-grid">
    <div class="tip-card">
      <span class="tip-icon">■</span>
      <h3>Suspicious URLs</h3>
      <p>Phishing links often mimic real brands with small changes: <em>paypa1.com</em>,
<em>amazon-secure-login.net</em>. Always inspect the full domain before clicking.</p>
    </div>
    <div class="tip-card">
      <span class="tip-icon">■</span>
      <h3>Urgent Email Requests</h3>
      <p>Attackers create fake urgency — "Your account will be suspended!" Legitimate companies never
demand immediate action via email links.</p>
    </div>
    <div class="tip-card">
      <span class="tip-icon">■</span>
      <h3>HTTPS Is Not Enough</h3>
      <p>A padlock icon only means the connection is encrypted, NOT that the site is trustworthy.
Phishing sites use HTTPS too.</p>
    </div>
    <div class="tip-card">
      <span class="tip-icon">■</span>
      <h3>IP Address in URL</h3>
      <p>URLs like <em>http://192.168.1.1/login</em> are a red flag. Real websites use domain names, not
raw IP addresses.</p>
    </div>
    <div class="tip-card">
      <span class="tip-icon">@</span>
      <h3>The "@" Trick</h3>
      <p>A URL containing "@" redirects you to the part after it. <em>http://safe.com@evil.com</em>
actually takes you to <em>evil.com</em>.</p>
    </div>
    <div class="tip-card">
      <span class="tip-icon">■</span>
      <h3>Think Before You Click</h3>
      <p>When in doubt, navigate directly to the official website by typing the address yourself — never
follow links in unsolicited messages.</p>
    </div>
  </div>
</section>
```

■ Detection workflow

```
<section id="workflow" class="workflow-section">
  <div class="section-header compact">
    <p class="section-kicker">WORKFLOW</p>
    <h2 class="section-title">How the detection engine works.</h2>
  </div>

  <div class="workflow">
    <div class="wf-step">
      <div class="wf-circle">1</div>
      <div class="wf-label">URL Input</div>
      <div class="wf-desc">User enters any URL or domain</div>
    </div>
    <div class="wf-arrow">→</div>

    <div class="wf-step">
      <div class="wf-circle">2</div>
      <div class="wf-label">Normalize</div>
      <div class="wf-desc">Strip whitespace, add protocol if missing</div>
    </div>
    <div class="wf-arrow">→</div>

    <div class="wf-step wf-branch">
      <div class="wf-circle warn">3</div>
      <div class="wf-label">Blacklist Check</div>
      <div class="wf-desc">Instant 100% phishing if domain is known-bad</div>
    </div>
    <div class="wf-arrow">→</div>

    <div class="wf-step">
      <div class="wf-circle">4</div>
      <div class="wf-label">Feature Extraction</div>
      <div class="wf-desc">URL length, hyphens, dots, IP flag, @ symbol…</div>
    </div>
    <div class="wf-arrow">→</div>

    <div class="wf-step dual">
      <div class="wf-circle model">5a</div>
      <div class="wf-label">Logistic Regression</div>
      <div class="wf-desc">Calculates LR risk %</div>
    </div>
    <div class="wf-plus">+</div>
    <div class="wf-step dual">
      <div class="wf-circle model">5b</div>
      <div class="wf-label">Random Forest</div>
      <div class="wf-desc">Calculates RF risk %</div>
    </div>
    <div class="wf-arrow">→</div>

    <div class="wf-step">
      <div class="wf-circle">6</div>
      <div class="wf-label">Aggregate Score</div>
      <div class="wf-desc">Average of both model probabilities</div>
    </div>
    <div class="wf-arrow">→</div>

    <div class="wf-step result">
      <div class="wf-circle res">7</div>
      <div class="wf-label">Verdict</div>
      <div class="wf-desc">SAFE · SUSPICIOUS · PHISHING</div>
    </div>
  </div>
  <p class="wf-note">* If a domain is found in the blacklist at step 3, steps 4–6 are skipped and
PHISHING is returned immediately.</p>
</section>
```

■ Analyzer input

```
<section id="analyzer" class="analyzer-section">
  <div class="section-header compact">
    <p class="section-kicker">LIVE ANALYZER</p>
    <h2 class="section-title">Analyze one URL or a batch.</h2>
  </div>

  <div class="analyzer-card">
    <div class="input-row">
      <textarea id="urlInput" rows="4" aria-label="URLs to analyze"
                placeholder="Paste one URL per
line&#10;https://example.com&#10;https://another-example.com"
                autocomplete="off" spellcheck="false"></textarea>
      <button id="analyzeBtn" onclick="analyzeUrl()">
        <span id="btnText">Analyze URLs</span>
        <span id="btnSpinner" class="spinner hidden"></span>
      </button>
    </div>
    <p class="input-hint">Enter up to 100 URLs, one per line. Full URLs, bare domains, and IP-based
links are supported.</p>
```

■ Batch results table

```
    <div id="batchPanel" class="batch-panel hidden" aria-live="polite">
      <h3 id="batchHeading">Batch results</h3>
      <div class="table-scroll">
        <table class="results-table">
          <thead><tr><th>URL</th><th>Verdict</th><th>Risk</th><th>Logistic Regression</th><th>Random
Forest</th></tr></thead>
          <tbody id="batchResults"></tbody>
        </table>
      </div>
    </div>
```

■ Single-URL result details

```
      <div id="resultPanel" class="result-panel hidden">
        <div id="verdictBadge" class="verdict-badge">
          <span id="verdictIcon"></span>
          <span id="verdictText"></span>
        </div>

        <div class="score-section">
          <div class="score-label">
            <span>Combined Risk Score</span>
            <strong id="combinedPct">—</strong>
          </div>
          <div class="bar-track">
            <div id="combinedBar" class="bar-fill combined"></div>
          </div>
        </div>

        <div class="models-grid">
          <div class="model-card" id="lrCard">
            <div class="model-header">
              <span class="model-icon">■</span>
              <span class="model-name">Logistic Regression</span>
              <span id="lrVerdict" class="model-verdict-badge"></span>
            </div>
            <div class="score-label">
              <span>LR Risk Score</span>
              <strong id="lrPct">—</strong>
            </div>
            <div class="bar-track">
              <div id="lrBar" class="bar-fill lr"></div>
            </div>
            <p id="lrDesc" class="model-desc">A linear probability model — fast, interpretable, and great
at spotting clear structural patterns.</p>
            <div class="model-reasons">
              <h5>Why LR flagged this:</h5>
              <ul id="lrReasons"></ul>
            </div>
          </div>

          <div class="model-card" id="rfCard">
            <div class="model-header">
              <span class="model-icon">■</span>
              <span class="model-name">Random Forest</span>
              <span id="rfVerdict" class="model-verdict-badge"></span>
            </div>
            <div class="score-label">
              <span>RF Risk Score</span>
              <strong id="rfPct">—</strong>
            </div>
            <div class="bar-track">
              <div id="rfBar" class="bar-fill rf"></div>
            </div>
            <p id="rfDesc" class="model-desc">An ensemble of decision trees — captures non-linear feature
interactions and subtle phishing patterns.</p>
            <div class="model-reasons">
              <h5>Why RF flagged this:</h5>
              <ul id="rfReasons"></ul>
            </div>
          </div>
        </div>

        <div class="reasons-section">
          <h4>Detection Insights</h4>
          <ul id="reasonsList"></ul>
        </div>
```

```
        </div>
      </div>
```

■ Model metrics and footer

```
      <section class="metrics-section" aria-labelledby="metricsTitle">
        <div class="metrics-heading">
          <div>
            <p class="section-kicker">MODEL EVALUATION</p>
            <h3 id="metricsTitle">Logistic Regression vs Random Forest</h3>
          </div>
          <span id="metricsMethod" class="metrics-method">Loading evaluation…</span>
        </div>
        <div class="table-scroll">
          <table class="results-table metrics-table">
<thead><tr><th>Model</th><th>Accuracy</th><th>Precision</th><th>Recall</th><th>F1-Score</th></tr></thead>
            <tbody id="metricsBody"></tbody>
          </table>
        </div>
        <p id="metricsNote" class="metrics-note"></p>
      </section>
    </section>
  </main>

  <footer id="about" class="site-footer">
    <p>WolfSecurity  |  Dual ML Engine  |  For educational &amp; research use only</p>
    <p class="footer-note">This tool does not replace professional cybersecurity advice. Always verify links
    through official channels.</p>
  </footer>

  <script src="/static/app.js"></script>
</body>
</html>
```

4. static/style.css — CSS Stylesheet

static/style.css CSS3

Reset and CSS variables

```
/* ============================================================
   PHISHING URL DETECTOR — Premium Landing Page Styles
   ============================================================ */

:root {
  --bg: #07111f;
  --bg-2: #0f1d32;
  --surface: rgba(15, 23, 42, 0.9);
  --surface-strong: #12213b;
  --card: rgba(15, 23, 42, 0.72);
  --card-soft: #12253d;
  --border: rgba(148, 163, 184, 0.18);
  --text: #edf4ff;
  --muted: #a5b5d2;
  --accent: #4fa3ff;
  --accent-2: #6d7cff;
  --safe: #34d399;
  --warning: #fbbf24;
  --danger: #f87171;
  --shadow: 0 18px 50px rgba(4, 10, 24, 0.42);
  --radius: 20px;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

html {
  scroll-behavior: smooth;
}

body {
  font-family: Inter, "Segoe UI", sans-serif;
  background:
    radial-gradient(circle at top, rgba(79, 163, 255, 0.2), transparent 35%),
    linear-gradient(180deg, #07111f 0%, #0a1324 100%);
  color: var(--text);
  line-height: 1.6;
}

a {
  color: inherit;
  text-decoration: none;
}

img {
  max-width: 100%;
  display: block;
}
```

■ Navigation

```
.topbar {
  position: sticky;
  top: 0;
  z-index: 20;
  display: flex;
  align-items: center;
  justify-content: space-between;
  max-width: 1180px;
  margin: 20px auto 0;
  padding: 14px 22px;
  background: rgba(9, 16, 28, 0.7);
  border: 1px solid var(--border);
  border-radius: 18px;
  backdrop-filter: blur(16px);
  box-shadow: 0 12px 30px rgba(2, 6, 23, 0.25);
}

.brand-wrap {
  display: flex;
  align-items: center;
  gap: 12px;
}

.brand-mark {
  width: 42px;
  height: 42px;
  border-radius: 14px;
  display: grid;
  place-items: center;
  background: linear-gradient(135deg, var(--accent), var(--accent-2));
  box-shadow: 0 10px 25px rgba(79, 163, 255, 0.4);
}

.brand-mark svg {
  width: 29px;
  height: 29px;
}

.wolf-head {
  fill: #f5f9ff;
}

.wolf-detail {
  fill: #17304c;
}

.wolf-eye {
  fill: #54d7ff;
}

.brand-text {
  display: flex;
  flex-direction: column;
  line-height: 1.1;
}

.brand-name {
  font-size: 1.02rem;
  font-weight: 800;
}

.brand-tag {
  color: var(--muted);
  font-size: 0.7rem;
```

```
  letter-spacing: 0.12rem;
  text-transform: uppercase;
}

.main-nav {
  display: flex;
  align-items: center;
  gap: 26px;
  color: var(--muted);
  font-size: 0.92rem;
}

.main-nav a {
  transition: color 0.2s ease;
}

.main-nav a:hover {
  color: var(--text);
}

.nav-cta,
.primary-btn,
.secondary-btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  border-radius: 12px;
  font-weight: 700;
  transition: transform 0.18s ease, opacity 0.18s ease;
}

.nav-cta,
.primary-btn {
  background: linear-gradient(135deg, var(--accent), var(--accent-2));
  box-shadow: 0 12px 30px rgba(79, 163, 255, 0.28);
  color: white;
}

.nav-cta {
  padding: 10px 18px;
  font-size: 0.9rem;
}

.primary-btn,
.secondary-btn {
  padding: 14px 22px;
}

.primary-btn:hover,
.secondary-btn:hover,
.nav-cta:hover,
.input-row button:hover {
  transform: translateY(-1px);
}

.secondary-btn {
  background: rgba(148, 163, 184, 0.08);
  border: 1px solid var(--border);
  color: var(--text);
}

.page-wrap {
  max-width: 1180px;
  margin: 0 auto;
  padding: 30px 22px 56px;
  display: flex;
  flex-direction: column;
```

```
  gap: 54px;
}
```

■ Hero and content sections

```
.hero {
  position: relative;
  overflow: hidden;
  border: 1px solid var(--border);
  border-radius: 30px;
  background: linear-gradient(135deg, rgba(8, 17, 31, 0.98), rgba(17, 31, 53, 0.85));
  box-shadow: var(--shadow);
}

.hero::before {
  content: "";
  position: absolute;
  inset: 0;
  background: radial-gradient(circle at 20% 10%, rgba(79, 163, 255, 0.22), transparent 28%);
}

.hero-grid {
  position: relative;
  z-index: 1;
  display: grid;
  grid-template-columns: 1.2fr 0.8fr;
  gap: 28px;
  padding: 54px 42px;
  align-items: center;
}

.eyebrow,
.section-kicker {
  margin-bottom: 12px;
  font-size: 0.72rem;
  letter-spacing: 0.16rem;
  color: var(--accent);
  font-weight: 800;
}

.hero-copy h1 {
  max-width: 620px;
  font-size: clamp(2.3rem, 4vw, 4rem);
  line-height: 1.08;
  letter-spacing: -0.05em;
  margin-bottom: 18px;
}

.hero-sub {
  max-width: 620px;
  color: var(--muted);
  font-size: 1.05rem;
}

.hero-actions {
  display: flex;
  align-items: center;
  gap: 16px;
  margin-top: 28px;
  flex-wrap: wrap;
}

.hero-stats {
  display: flex;
  align-items: center;
  gap: 14px;
  margin-top: 28px;
  flex-wrap: wrap;
}
```

```
.stat-box {
  min-width: 130px;
  display: flex;
  flex-direction: column;
  gap: 4px;
  padding: 14px 16px;
  border-radius: 14px;
  background: rgba(148, 163, 184, 0.06);
  border: 1px solid var(--border);
}

.stat-box strong {
  font-size: 1.1rem;
  color: var(--text);
}

.stat-box span {
  color: var(--muted);
  font-size: 0.78rem;
}

.hero-panel {
  display: flex;
  justify-content: center;
}

.preview-card {
  width: min(100%, 420px);
  background: rgba(10, 19, 33, 0.85);
  border: 1px solid var(--border);
  border-radius: 28px;
  padding: 18px 18px 20px;
  box-shadow: 0 18px 42px rgba(4, 6, 18, 0.42);
}

.preview-header {
  display: flex;
  gap: 8px;
  margin-bottom: 18px;
}

.dot {
  width: 12px;
  height: 12px;
  border-radius: 50%;
  display: block;
}

.dot.green { background: #37d39a; }
.dot.yellow { background: #fbbf24; }
.dot.red { background: #f87171; }

.scan-box {
  display: flex;
  flex-direction: column;
  gap: 10px;
  padding: 16px 18px;
  border-radius: 16px;
  background: rgba(79, 163, 255, 0.08);
  border: 1px solid rgba(79, 163, 255, 0.2);
}

.scan-label {
  color: var(--muted);
  font-size: 0.72rem;
  letter-spacing: 0.08rem;
  text-transform: uppercase;
```

```
}

.scan-box strong {
  font-size: 1.22rem;
  color: var(--text);
}

.scan-score-row {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-top: 18px;
  color: var(--muted);
  font-size: 0.93rem;
}

.scan-score-row strong {
  color: #fca5a5;
  font-size: 1.8rem;
}

.mini-bar {
  height: 12px;
  margin-top: 12px;
  background: rgba(148, 163, 184, 0.08);
  border-radius: 999px;
  overflow: hidden;
}

.mini-bar span {
  display: block;
  width: 100%;
  height: 100%;
  border-radius: 999px;
  background: linear-gradient(90deg, #f87171, #fbbf24, #34d399);
}

.mini-list {
  margin-top: 18px;
  list-style: none;
  display: flex;
  flex-direction: column;
  gap: 10px;
  color: var(--muted);
  font-size: 0.9rem;
}

.mini-list li {
  display: flex;
  align-items: center;
  gap: 10px;
}

.mini-list li::before {
  content: "";
  width: 8px;
  height: 8px;
  border-radius: 50%;
  background: linear-gradient(135deg, var(--safe), var(--warning));
  box-shadow: 0 0 10px rgba(52, 211, 153, 0.4);
}

.awareness-banner,
.workflow-section,
.analyzer-section {
  display: flex;
  flex-direction: column;
```

```
  gap: 24px;
}

.section-header {
  display: flex;
  flex-direction: column;
  gap: 5px;
}

.section-header.compact {
  margin-bottom: 6px;
}

.section-title {
  font-size: clamp(1.8rem, 3vw, 2.3rem);
  line-height: 1.2;
  color: var(--text);
}

.awareness-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 18px;
}

.tip-card {
  background: rgba(15, 24, 39, 0.84);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  padding: 22px 20px;
  box-shadow: 0 12px 28px rgba(1, 5, 15, 0.22);
  transition: transform 0.2s ease, border-color 0.2s ease;
}

.tip-card:hover {
  transform: translateY(-3px);
  border-color: rgba(79, 163, 255, 0.35);
}

.tip-icon {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: 46px;
  height: 46px;
  border-radius: 14px;
  background: rgba(79, 163, 255, 0.08);
  font-size: 1.5rem;
  margin-bottom: 14px;
}

.tip-card h3 {
  font-size: 1.1rem;
  margin-bottom: 8px;
}

.tip-card p {
  color: var(--muted);
  font-size: 0.95rem;
}

.tip-card em {
  color: #f8d87f;
  font-style: normal;
  font-weight: 700;
}
.workflow {
```

```
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 10px;
  padding: 26px 18px;
  background: rgba(15, 24, 39, 0.82);
  border: 1px solid var(--border);
  border-radius: 24px;
}

.wf-step {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  min-width: 95px;
  max-width: 110px;
  text-align: center;
  flex: 1 1 auto;
}

.wf-circle {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 48px;
  height: 48px;
  border-radius: 50%;
  background: linear-gradient(135deg, var(--accent), var(--accent-2));
  color: white;
  font-weight: 800;
  margin-bottom: 8px;
}

.wf-circle.warn { background: linear-gradient(135deg, #f59e0b, #f97316); }
.wf-circle.model { background: linear-gradient(135deg, #8b5cf6, #6366f1); }
.wf-circle.res { background: linear-gradient(135deg, #34d399, #10b981); }

.wf-label {
  font-size: 0.78rem;
  font-weight: 700;
}

.wf-desc {
  margin-top: 4px;
  color: var(--muted);
  font-size: 0.62rem;
  line-height: 1.4;
}

.wf-arrow,
.wf-plus {
  font-size: 1.6rem;
  color: #8bbdff;
  font-weight: 700;
}

.wf-note {
  color: var(--muted);
  font-size: 0.82rem;
  font-style: italic;
}
```

■ Analyzer controls

```
.analyzer-card {
  background: rgba(15, 24, 39, 0.9);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  padding: 28px 22px;
  box-shadow: var(--shadow);
}

.input-row {
  display: flex;
  gap: 12px;
  flex-wrap: wrap;
}

.input-row textarea {
  flex: 1 1 560px;
  min-width: 220px;
  min-height: 118px;
  resize: vertical;
  padding: 14px 18px;
  border-radius: 14px;
  border: 1px solid var(--border);
  background: rgba(8, 12, 22, 0.72);
  color: var(--text);
  font-size: 1rem;
  outline: none;
}

.input-row textarea::placeholder {
  color: rgba(148, 163, 184, 0.8);
}

.input-row textarea:focus {
  border-color: rgba(79, 163, 255, 0.8);
  box-shadow: 0 0 0 4px rgba(79, 163, 255, 0.08);
}

.input-row button {
  padding: 15px 24px;
  border: none;
  border-radius: 14px;
  background: linear-gradient(135deg, var(--accent), var(--accent-2));
  color: white;
  font-weight: 800;
  cursor: pointer;
  display: inline-flex;
  align-items: center;
  gap: 10px;
}

.input-hint {
  margin-top: 10px;
  color: var(--muted);
  font-size: 0.78rem;
}
```

■ Batch and model-metric tables

```
.batch-panel,
.metrics-section {
  margin-top: 28px;
  padding: 20px;
  border: 1px solid var(--border);
  border-radius: 16px;
  background: rgba(10, 16, 28, 0.72);
}

.batch-panel h3,
.metrics-heading h3 {
  margin-bottom: 14px;
  color: var(--text);
  font-size: 1rem;
}

.metrics-heading {
  display: flex;
  align-items: end;
  justify-content: space-between;
  gap: 16px;
  margin-bottom: 12px;
}

.metrics-heading .section-kicker {
  margin-bottom: 4px;
}

.metrics-heading h3 {
  margin-bottom: 0;
}

.metrics-method,
.metrics-note {
  color: var(--muted);
  font-size: 0.78rem;
}

.metrics-note {
  margin-top: 12px;
  line-height: 1.5;
}

.table-scroll {
  overflow-x: auto;
}

.results-table {
  width: 100%;
  border-collapse: collapse;
  text-align: left;
  font-size: 0.82rem;
}

.results-table th,
.results-table td {
  padding: 11px 12px;
  border-bottom: 1px solid var(--border);
}

.results-table th {
  color: var(--muted);
  font-size: 0.7rem;
  font-weight: 700;
```

```
  text-transform: uppercase;
}

.results-table td:first-child {
  max-width: 380px;
  overflow-wrap: anywhere;
}

.table-verdict {
  font-weight: 800;
}

.verdict-text-SAFE { color: var(--safe); }
.verdict-text-SUSPICIOUS { color: var(--warning); }
.verdict-text-PHISHING { color: var(--danger); }

.metrics-section {
  margin-top: 22px;
}
```

■ Single-URL result panel

```
.result-panel {
  margin-top: 28px;
  animation: reveal 0.3s ease;
}

@keyframes reveal {
  from {
    opacity: 0;
    transform: translateY(12px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

.verdict-badge {
  display: inline-flex;
  align-items: center;
  gap: 12px;
  padding: 12px 18px;
  border-radius: 999px;
  border: 1px solid var(--border);
  font-weight: 800;
  letter-spacing: 0.06rem;
  text-transform: uppercase;
  margin-bottom: 22px;
}

.verdict-SAFE {
  background: rgba(52, 211, 153, 0.08);
  border-color: rgba(52, 211, 153, 0.3);
  color: #8cf1c7;
}

.verdict-SUSPICIOUS {
  background: rgba(251, 191, 36, 0.08);
  border-color: rgba(251, 191, 36, 0.3);
  color: #f9d97c;
}

.verdict-PHISHING {
  background: rgba(248, 113, 113, 0.08);
  border-color: rgba(248, 113, 113, 0.3);
  color: #fca5a5;
}

.score-section {
  margin-bottom: 24px;
}

.score-label {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 8px;
  margin-bottom: 8px;
  color: var(--muted);
  font-size: 0.82rem;
}

.score-label strong {
  font-size: 1.05rem;
  color: var(--text);
```

```
}

.bar-track {
  height: 12px;
  border-radius: 999px;
  overflow: hidden;
  background: rgba(148, 163, 184, 0.09);
}

.bar-fill {
  width: 0;
  height: 100%;
  border-radius: inherit;
  transition: width 0.9s ease;
}

.bar-fill.combined {
  background: linear-gradient(90deg, #34d399, #fbbf24, #f87171);
}

.bar-fill.lr {
  background: linear-gradient(90deg, #60a5fa, #8b5cf6);
}

.bar-fill.rf {
  background: linear-gradient(90deg, #34d399, #22c55e);
}
```

■ Model and reason cards

```
.models-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 16px;
  margin-bottom: 24px;
}

.model-card {
  background: rgba(10, 16, 28, 0.9);
  border: 1px solid var(--border);
  border-radius: 18px;
  padding: 18px 16px;
}

.model-header {
  display: flex;
  align-items: center;
  gap: 8px;
  margin-bottom: 12px;
  flex-wrap: wrap;
}

.model-icon {
  font-size: 1.2rem;
}

.model-name {
  flex: 1;
  font-weight: 700;
}

.model-verdict-badge {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  font-size: 0.64rem;
  padding: 4px 8px;
  border-radius: 999px;
  font-weight: 800;
  text-transform: uppercase;
}

.mvb-SAFE {
  background: rgba(52, 211, 153, 0.12);
  color: #8cf1c7;
}

.mvb-SUSPICIOUS {
  background: rgba(251, 191, 36, 0.12);
  color: #f9d97c;
}

.mvb-PHISHING {
  background: rgba(248, 113, 113, 0.12);
  color: #fca5a5;
}

.model-desc {
  margin-top: 12px;
  color: var(--muted);
  font-size: 0.78rem;
}

.model-reasons {
```

```
  margin-top: 16px;
  padding-top: 12px;
  border-top: 1px solid var(--border);
}

.model-reasons h5 {
  margin-bottom: 8px;
  color: #dbe7ff;
  font-size: 0.7rem;
  text-transform: uppercase;
  letter-spacing: 0.08rem;
}

.model-reasons ul,
.reasons-section ul {
  list-style: none;
  display: flex;
  flex-direction: column;
  gap: 8px;
}

.model-reason-item,
.reasons-section li {
  border-left: 3px solid rgba(79, 163, 255, 0.8);
  background: rgba(148, 163, 184, 0.04);
  padding: 8px 10px;
  border-radius: 0 10px 10px 0;
  color: var(--muted);
  font-size: 0.74rem;
  line-height: 1.5;
}

.reason-SAFE { border-left-color: var(--safe); }
.reason-SUSPICIOUS { border-left-color: var(--warning); }
.reason-PHISHING { border-left-color: var(--danger); }

.reasons-section {
  background: rgba(10, 16, 28, 0.9);
  border: 1px solid var(--border);
  border-radius: 18px;
  padding: 18px 16px;
}

.reasons-section h4 {
  font-size: 0.9rem;
  text-transform: uppercase;
  letter-spacing: 0.08rem;
  color: #dfe9ff;
  margin-bottom: 12px;
}
```

■ Footer

```
.site-footer {
  text-align: center;
  padding: 18px 20px 38px;
  color: var(--muted);
  font-size: 0.78rem;
  line-height: 1.8;
}

.footer-note {
  font-size: 0.7rem;
  opacity: 0.72;
}

.hidden {
  display: none !important;
}

.spinner {
  width: 16px;
  height: 16px;
  border: 2px solid rgba(255, 255, 255, 0.35);
  border-top-color: rgba(255, 255, 255, 0.95);
  border-radius: 50%;
  animation: spin 0.8s linear infinite;
}

@keyframes spin {
  to {
    transform: rotate(360deg);
  }
}
```

■ Responsive breakpoints

```
@media (max-width: 900px) {
  .hero-grid {
    grid-template-columns: 1fr;
    padding: 36px 24px;
  }

  .main-nav {
    display: none;
  }
}

@media (max-width: 640px) {
  .topbar {
    margin-top: 12px;
    padding: 12px 16px;
  }

  .nav-cta {
    padding: 9px 12px;
    font-size: 0.8rem;
  }

  .hero-actions {
    flex-direction: column;
    align-items: stretch;
  }

  .primary-btn,
  .secondary-btn {
    width: 100%;
  }

  .hero-stats {
    display: grid;
    grid-template-columns: repeat(3, minmax(0, 1fr));
  }

  .models-grid {
    grid-template-columns: 1fr;
  }

  .metrics-heading {
    align-items: flex-start;
    flex-direction: column;
  }

  .results-table {
    min-width: 620px;
  }

  .workflow {
    padding: 18px 14px;
    gap: 8px;
  }

  .wf-step {
    min-width: 70px;
    max-width: 85px;
  }

  .wf-desc {
    display: none;
  }
}
```

5. static/app.js — JavaScript Frontend Logic

■ static/app.js JavaScript ES2020

■ Risk and DOM helper functions

```
/* ============================================================
   Phishing URL Detector — Frontend Logic
   ============================================================ */

function classifyRisk(pct) {
  if (pct < 35) return "SAFE";
  if (pct < 70) return "SUSPICIOUS";
  return "PHISHING";
}

function verdictIcon(status) {
  return { SAFE: "■", SUSPICIOUS: "■■", PHISHING: "■" }[status] ?? "■";
}

function setBar(barId, pct) {
  const el = document.getElementById(barId);
  // Use setTimeout so CSS transition triggers after element is visible
  setTimeout(() => { el.style.width = Math.min(pct, 100) + "%"; }, 50);
}

function populateReasons(listId, reasons, status) {
  const ul = document.getElementById(listId);
  ul.innerHTML = "";
  reasons.forEach(r => {
    const li = document.createElement("li");
    li.textContent = r;
    li.className = "model-reason-item reason-" + status;
    ul.appendChild(li);
  });
}
```

■ URL analysis request

```
async function analyzeUrl() {
  const input  = document.getElementById("urlInput");
  const btn    = document.getElementById("analyzeBtn");
  const btnTxt = document.getElementById("btnText");
  const spin   = document.getElementById("btnSpinner");
  const panel  = document.getElementById("resultPanel");

  const urls = input.value.split(/\r?\n/).map(url => url.trim()).filter(Boolean);
  if (!urls.length) { input.focus(); input.style.borderColor = "#ef4444"; setTimeout(() =>
    input.style.borderColor = "", 1200); return; }

  // Loading state
  btn.disabled = true;
  btnTxt.textContent = "Analyzing…";
  spin.classList.remove("hidden");
  panel.classList.add("hidden");
  document.getElementById("batchPanel").classList.add("hidden");

  try {
    const res  = await fetch("/predict", {
      method:  "POST",
      headers: { "Content-Type": "application/json" },
      body:    JSON.stringify(urls.length === 1 ? { url: urls[0] } : { urls })
    });
    const data = await res.json();
    if (data.error) throw new Error(data.error);
    if (data.results) renderBatchResults(data);
    else renderResult(data);
  } catch (err) {
    alert("Error: " + err.message);
  } finally {
    btn.disabled = false;
    btnTxt.textContent = "Analyze URLs";
    spin.classList.add("hidden");
  }
}
```

■ Batch result rendering

```
function renderBatchResults(data) {
  const panel = document.getElementById("batchPanel");
  const tbody = document.getElementById("batchResults");
  tbody.replaceChildren();
  document.getElementById("batchHeading").textContent = `Results for ${data.count} URLs`;

  data.results.forEach(result => {
    const row = document.createElement("tr");
    [result.url, result.status, `${result.risk_score}%`,
      result.lr_prob === null ? "Blacklist" : `${result.lr_status} · ${result.lr_prob}%`,
      result.rf_prob === null ? "Blacklist" : `${result.rf_status} · ${result.rf_prob}%`
    ].forEach((value, index) => {
      const cell = document.createElement("td");
      cell.textContent = value;
      if (index === 1) cell.className = "table-verdict verdict-text-" + result.status;
      row.appendChild(cell);
    });
    tbody.appendChild(row);
  });

  panel.classList.remove("hidden");
  panel.scrollIntoView({ behavior: "smooth", block: "nearest" });
}
```

■ Model metrics loading

```
async function loadModelMetrics() {
  const body = document.getElementById("metricsBody");
  const method = document.getElementById("metricsMethod");
  const note = document.getElementById("metricsNote");
  try {
    const response = await fetch("/metrics");
    const data = await response.json();
    if (!response.ok) throw new Error(data.error || "Unable to load model metrics.");
    Object.entries(data.models).forEach(([name, metrics]) => {
      const row = document.createElement("tr");
      [name, metrics.accuracy, metrics.precision, metrics.recall, metrics.f1_score]
        .forEach((value, index) => {
          const cell = document.createElement("td");
          cell.textContent = index === 0 ? value : `${value}%`;
          row.appendChild(cell);
        });
      body.appendChild(row);
    });
    method.textContent = data.evaluation;
    note.textContent = `Scores use phishing as the positive class and are based on ${data.sample_count}
    synthetic training URLs. Treat this small demo dataset as illustrative, not a measure of production
    performance.`;
  } catch (error) {
    method.textContent = "Evaluation unavailable";
    note.textContent = error.message;
  }
}
```

■ Single-result rendering and events

```
function renderResult(data) {
  const panel = document.getElementById("resultPanel");

  /* --- Overall verdict --- */
  const badge = document.getElementById("verdictBadge");
  badge.className = "verdict-badge verdict-" + data.status;
  document.getElementById("verdictIcon").textContent = verdictIcon(data.status);
  document.getElementById("verdictText").textContent = data.status;

  /* --- Combined score --- */
  document.getElementById("combinedPct").textContent = data.risk_score + "%";
  setBar("combinedBar", data.risk_score);

  /* --- Blacklist path: no per-model data --- */
  const modelsGrid = document.querySelector(".models-grid");
  if (data.lr_prob == null) {
    modelsGrid.style.display = "none";
  } else {
    modelsGrid.style.display = "grid";

    /* LR model */
    const lrStatus = classifyRisk(data.lr_prob);
    document.getElementById("lrPct").textContent          = data.lr_prob + "%";
    document.getElementById("lrVerdict").textContent      = lrStatus;
    document.getElementById("lrVerdict").className        = "model-verdict-badge mvb-" + lrStatus;
    setBar("lrBar", data.lr_prob);
    populateReasons("lrReasons", data.lr_reasons || [], lrStatus);

    /* RF model */
    const rfStatus = classifyRisk(data.rf_prob);
    document.getElementById("rfPct").textContent          = data.rf_prob + "%";
    document.getElementById("rfVerdict").textContent      = rfStatus;
    document.getElementById("rfVerdict").className        = "model-verdict-badge mvb-" + rfStatus;
    setBar("rfBar", data.rf_prob);
    populateReasons("rfReasons", data.rf_reasons || [], rfStatus);
  }

  /* --- Reasons --- */
  const list = document.getElementById("reasonsList");
  list.innerHTML = "";
  (data.reasons || []).forEach(r => {
    const li = document.createElement("li");
    li.textContent = r;
    list.appendChild(li);
  });

  panel.classList.remove("hidden");
  panel.scrollIntoView({ behavior: "smooth", block: "nearest" });
}

/* Allow pressing Enter in the input */
document.addEventListener("DOMContentLoaded", () => {
  document.getElementById("urlInput").addEventListener("keydown", event => {
    if (event.key === "Enter" && (event.ctrlKey || event.metaKey)) analyzeUrl();
  });
  loadModelMetrics();
});
```

6. How to Run the Application

Prerequisites

Python	3.10 or later (tested on 3.13)
Install	pip install flask scikit-learn pandas numpy

Start the server

```
# From the d:\safi directory:
python phishing_detector.py
```

Open in browser

Navigate to **http://127.0.0.1:5000** — the app trains both ML models, runs a self-test, then serves the full web UI.

cURL API example

```
curl -X POST http://127.0.0.1:5000/predict \
     -H "Content-Type: application/json" \
     -d '{"url": "http://login-secure-paypal-verification.com"}'
```

Analyze a URL batch

```
curl -X POST http://127.0.0.1:5000/predict \
     -H "Content-Type: application/json" \
     -d '{"urls": ["http://suspicious-site.com", "https://www.example.com"]}'
```

Compare model evaluation metrics

```
curl http://127.0.0.1:5000/metrics
```

The metrics use phishing as the positive class and the project's 20 synthetic URLs. They illustrate the demo only and should not be treated as production performance estimates.

Sample JSON response

```
{
  "status":      "PHISHING",
  "risk_score":  100.0,
  "lr_prob":     100.0,
  "lr_status":   "PHISHING",
  "lr_reasons":  [
    "LR heavily weighted the 3 hyphens — strong positive coefficient for phishing.",
    "URL length (50 chars) pushed the LR score up — longer URLs correlate with obfuscation."
  ],
  "rf_prob":     100.0,
  "rf_status":   "PHISHING",
  "rf_reasons":  [
    "RF trees found 3 hyphens AND long URL together — strong phishing interaction pattern.",
    "Near-unanimous agreement across the ensemble."
  ],
  "reasons":     [
    "Multiple hyphens in the domain — common trick to mimic legitimate brands.",
    "Abnormally long URL (50 characters) — often used to hide the real destination."
  ]
}
```